АЙСМАРТ

Освоение образовательной платформы и правил обучения

Занятие 1. Освоение образовательной платформы и правил обучения

Содержание

- Введение
- Что вы узнаете на этом занятии
- 1. О чём этот курс
- Чему вы научитесь
- 2. Как построено обучение
- Почему важно проходить темы по порядку
- 3. Как проходит обучение
- Структура уроков
- Обратная связь
- Если возникли вопросы
- 4. Среда разработки Jupyter Notebook
- Почему именно Jupyter Notebook?
- Что такое ноутбук?
- Из чего состоит ноутбук
- Ячейки — основа Jupyter Notebook
- Ячейка Code
- Ячейка Markdown
- Выполнение кода
- Что означают In и Out?
- Что такое Kernel?
- Как правильно сохранять работу
- Полезные рекомендации

- Мини-проверка
- Итоги занятия

Введение

Сегодня искусственный интеллект перестал быть технологией будущего — он уже помогает врачам ставить диагнозы, переводит тексты, управляет беспилотными автомобилями, рекомендует фильмы и музыку, анализирует огромные объёмы данных и автоматизирует работу предприятий.

При этом искусственный интеллект не заменяет необходимость понимать задачу и проверять результат. Любая интеллектуальная система работает по определённым правилам, использует данные и может допускать ошибки.

Однако любой искусственный интеллект начинается не с магии, а с человека, который умеет анализировать данные, писать программы и понимать, как работают современные технологии.

Именно этому посвящён данный курс.

Во время обучения вы познакомитесь с языком программирования Python, научитесь работать с данными, узнаете, как создаются интеллектуальные системы, познакомитесь с основами машинного обучения и разработаете собственный учебный проект с использованием технологий искусственного интеллекта.

Новые навыки будут осваиваться постепенно: сначала основные инструменты и команды, затем работа с данными, после этого — принципы машинного обучения и проектная деятельность.

Сегодняшнее занятие отличается от остальных.

Мы пока не будем писать программы и создавать модели искусственного интеллекта. Главная задача первого урока — познакомиться с курсом, разобраться, как будет организовано обучение, изучить образовательную платформу и узнать, в какой программе будут выполняться все практические задания.

Такое знакомство необходимо, чтобы во время следующих занятий сосредоточиться на содержании задач, а не тратить время на поиск материалов, сохранение файлов и управление рабочей средой.

После этого вы будете полностью готовы к началу изучения программирования.

Что вы узнаете на этом занятии

После изучения материала вы сможете:

- понимать, как организовано обучение на курсе;
- ориентироваться в структуре образовательной платформы;
- знать, как проходят занятия и проверка работ;
- понимать требования к выполнению заданий;

- узнать, в какой программе будут выполняться практические работы;
- подготовиться к дальнейшему изучению Python и искусственного интеллекта.

Особое внимание будет уделено Jupyter Notebook: типам ячеек, порядку выполнения команд, работе ядра и правилам сохранения ноутбука.

1. О чём этот курс

Искусственный интеллект — одно из самых быстро развивающихся направлений современной информатики.

Под искусственным интеллектом понимают технологии, которые позволяют компьютерным системам решать задачи, обычно требующие анализа информации: распознавать объекты, находить закономерности, строить прогнозы или выбирать подходящий вариант действия.

Сегодня технологии искусственного интеллекта применяются практически во всех сферах жизни:

- медицине;
- промышленности;
- банковской сфере;
- образовании;
- транспорте;
- науке;
- разработке компьютерных игр;
- интернет-сервисах.

Практически каждый современный цифровой сервис использует технологии искусственного интеллекта.

При этом искусственный интеллект может быть только одной из частей сервиса. Например, интернет-магазин включает каталог, систему оплаты и личный кабинет, а интеллектуальный алгоритм может отвечать лишь за рекомендации товаров.

Например:

- голосовые помощники распознают речь;
- навигаторы строят оптимальные маршруты;
- интернет-магазины рекомендуют товары;
- социальные сети подбирают интересный контент;
- системы безопасности распознают лица;
- чат-боты помогают пользователям получать ответы на вопросы.

Во всех этих примерах система получает входную информацию, обрабатывает её и формирует результат. Качество результата зависит от используемого алгоритма, данных и правильности постановки задачи.

Чтобы создавать подобные системы, недостаточно знать только программирование.

Необходимо понимать:

- как работают данные;
- каким образом компьютер принимает решения;
- почему модели иногда ошибаются;
- как оценивать качество работы алгоритмов;
- как безопасно использовать технологии искусственного интеллекта.

Например, программа может быть написана без технических ошибок, но давать неточные результаты из-за неполных или неверно подготовленных данных. Поэтому работа над интеллектуальной системой включает не только создание кода, но и проверку исходной информации и полученных выводов.

Именно этим вопросам посвящён данный курс.

Важно

Искусственный интеллект не гарантирует правильный ответ в каждой ситуации. Результаты работы модели необходимо оценивать и проверять, особенно если от них зависит важное решение.

Чему вы научитесь

В процессе обучения вы постепенно освоите несколько важных направлений.

Эти направления связаны между собой: программа обрабатывает данные, алгоритм находит в них закономерности, а разработчик оценивает полученный результат и объясняет его.

Программирование на Python

Вы изучите один из самых популярных языков программирования в мире.

Python позволяет записывать алгоритмы в виде понятной последовательности команд. Знание синтаксиса языка поможет объяснить компьютеру, какие данные нужно получить, как их обработать и в каком виде представить результат.

На Python создаются:

- сайты;
- игры;
- научные исследования;
- системы анализа данных;
- программы искусственного интеллекта.

Один и тот же язык может применяться для разных задач, однако конкретные инструменты и библиотеки выбираются в зависимости от цели проекта.

Работа с данными

Практически любой искусственный интеллект начинает работу с анализа данных.

Данные могут содержать числа, тексты, категории и другие сведения. Перед использованием необходимо проверить их полноту, формат и корректность, поскольку ошибки в исходной информации влияют на дальнейшие вычисления.

Вы научитесь:

- открывать таблицы;
- находить ошибки;
- обрабатывать информацию;
- выполнять вычисления;
- строить графики;
- делать выводы на основе полученных результатов.

Например, при анализе таблицы сначала можно проверить названия столбцов и пропущенные значения, затем выполнить вычисления, а после этого представить результаты на графике. Такая последовательность уменьшает вероятность сделать вывод на основе некорректных данных.

Основы искусственного интеллекта

Вы узнаете:

- что такое машинное обучение;
- чем отличается классификация от прогнозирования;
- как компьютер обучается на примерах;
- почему модели могут ошибаться;
- каким образом оценивается качество работы модели.

При классификации система выбирает один из заранее определённых классов, а при прогнозировании может вычислять ожидаемое числовое значение. В обоих случаях недостаточно просто получить ответ: необходимо проверить, насколько хорошо выбранный метод справляется с задачей.

Создание собственного проекта

Завершающей частью курса станет разработка собственного учебного проекта.

Проект объединит отдельные навыки в общий процесс: от формулировки задачи до представления результата. Важно будет не только получить работающую программу, но и объяснить выбранный способ решения.

В ходе работы необходимо будет:

- определить задачу;
- подготовить данные;
- выбрать подходящий способ решения;
- проверить качество результата;
- оформить проект;
- представить его на защите.

Каждый этап влияет на следующий. Например, неточная формулировка задачи затруднит выбор способа решения, а неподготовленные данные снизят качество результата.

Именно такой подход используется при создании большинства современных цифровых продуктов.

Важно

На этом курсе вы будете не только изучать теорию.

Основная цель обучения — научиться самостоятельно применять полученные знания при решении практических задач.

Поэтому большая часть занятий будет посвящена выполнению практических заданий, анализу данных и созданию собственного проекта.

При выполнении практической работы важно уметь объяснить последовательность действий и проверить результат, а не только получить готовый ответ.

2. Как построено обучение

Все темы курса расположены в определённой последовательности. Каждый следующий раздел опирается на знания, полученные ранее. Именно поэтому рекомендуется проходить обучение последовательно и не пропускать занятия.

Если отдельная тема вызывает затруднения, полезно вернуться к предыдущему материалу и проверить, все ли основные понятия и действия понятны. Такой возврат является обычной частью обучения.

Структура курса выглядит следующим образом.

Модуль 1

Основы программирования на Python и знакомство с рабочими инструментами.

На этом этапе вы научитесь писать первые программы и познакомитесь со средой разработки.

Навыки работы с файлами, ячейками и запуском команд понадобятся во всех последующих модулях.

Модуль 2

Работа с данными.

Вы научитесь читать таблицы, анализировать информацию и подготавливать данные для дальнейшей обработки.

На этом этапе особенно важно соблюдать последовательность действий: сначала изучить структуру данных, затем проверить их и только после этого выполнять анализ.

Модуль 3

Основы искусственного интеллекта и машинного обучения.

Вы познакомитесь с принципами построения интеллектуальных систем и научитесь понимать, каким образом компьютер принимает решения.

Знание программирования и способов анализа данных позволит сосредоточиться на логике обучения и проверки моделей.

Модуль 4

Проектная деятельность.

В завершение курса вы создадите собственный учебный проект, используя знания, полученные на предыдущих занятиях.

Проект покажет, как отдельные темы курса соединяются при решении одной законченной задачи.

Почему важно проходить темы по порядку

Изучение программирования похоже на строительство дома. Сначала создаётся фундамент. Затем строятся стены. После этого появляется крыша. Если попытаться построить крышу без фундамента, дом не получится.

Точно так же устроено и обучение программированию.

Например, невозможно:

- работать с данными, не зная языка Python;
- создавать модели машинного обучения, не умея анализировать данные;
- выполнить итоговый проект без понимания предыдущих тем.

Даже если отдельное задание удастся выполнить по готовому образцу, без понимания предыдущих шагов будет сложно исправить ошибку или применить тот же способ к новой задаче.

Поэтому рекомендуется выполнять задания последовательно и внимательно изучать каждый новый раздел.

Перед переходом к следующей теме полезно проверить, удаётся ли самостоятельно повторить

основные действия текущего занятия и объяснить их назначение.

3. Как проходит обучение

Все материалы курса размещаются на образовательной платформе.

Именно здесь будет проходить практически вся ваша работа.

Образовательная платформа объединяет материалы, задания и обратную связь. Поэтому важно использовать её систематически и проверять, какие работы уже выполнены, а какие ещё ожидают отправки или исправления.

На платформе вы сможете:

- открывать новые занятия;
- изучать теоретический материал;
- выполнять практические задания;
- проходить тестирование;
- отправлять работы преподавателю;
- получать обратную связь;
- отслеживать собственный прогресс.

После отправки работы рекомендуется убедиться, что на платформе появился загруженный файл или соответствующий статус задания. Это помогает избежать ситуации, когда выполненная работа осталась только на компьютере и не была передана преподавателю.

Практически весь курс построен таким образом, чтобы вы могли постепенно двигаться от простых заданий к более сложным.

Структура уроков

Практически каждое занятие курса построено одинаково.

Обычно урок состоит из нескольких частей:

1. введение;
2. изучение новой темы;
3. примеры;
4. практические задания (начиная со следующих занятий);
5. подведение итогов.

Такая структура позволяет постепенно переходить от изучения новой информации к её практическому применению.

Во введении определяется задача занятия, в теоретической части рассматриваются новые понятия, а примеры показывают способы их применения. Практические задания позволяют

проверить понимание материала, а итоги помогают выделить основные результаты.

Если при выполнении практической части возникло затруднение, следует определить, к какому понятию или примеру относится задача, и вернуться к соответствующей части урока.

Обратная связь

Во время обучения ваши работы будут проверяться преподавателем.

После проверки вы можете получить:

- комментарии;
- рекомендации;
- замечания;
- советы по исправлению ошибок.

Обратная связь является важной частью обучения.

Она помогает понять, какие темы уже освоены хорошо, а над какими ещё необходимо поработать.

Полученное замечание следует сопоставить с конкретным местом задания, затем внести исправление и повторно проверить работу. Важно исправлять не только внешний результат, но и причину ошибки.

Обратите внимание

Комментарий преподавателя не всегда содержит готовое решение. Его задача может состоять в том, чтобы указать направление поиска и помочь самостоятельно обнаружить ошибку.

Если возникли вопросы

Во время обучения могут возникать различные вопросы.

Например:

- программа работает не так, как ожидалось;
- появилось сообщение об ошибке;
- непонятно объяснение новой темы;
- возникли сложности при выполнении задания.

В подобных ситуациях не стоит пытаться решить проблему случайным способом.

Сначала полезно определить, на каком именно шаге возникла проблема: при открытии файла, запуске ячейки, вводе команды или получении результата. Чем точнее описана ситуация, тем легче найти её причину.

Лучше внимательно изучить комментарии преподавателя, перечитать материал занятия и при необходимости обратиться за помощью через образовательную платформу.

При обращении за помощью следует указать номер задания, описать выполненные действия и приложить точный текст ошибки или снимок экрана. Формулировка «программа не работает» содержит недостаточно информации для поиска причины.

Важно

Не бойтесь допускать ошибки.

Ошибки являются естественной частью обучения программированию.

Практически каждый программист ежедневно сталкивается с ошибками в коде. Главное — научиться спокойно анализировать проблему, находить её причину и исправлять её.

После исправления необходимо снова выполнить соответствующую ячейку и проверить, изменился ли результат ожидаемым образом.

4. Среда разработки Jupyter Notebook

Практически все практические задания курса вы будете выполнять в программе

Jupyter Notebook

Это одна из самых популярных сред разработки в области анализа данных, машинного обучения и искусственного интеллекта. Jupyter Notebook используют студенты университетов, научные сотрудники, аналитики данных, инженеры машинного обучения и разработчики программного обеспечения по всему миру.

Среда разработки предоставляет инструменты для создания, запуска и проверки программ. В Jupyter работа организована не как единый длинный файл, а как последовательность отдельных ячеек.

Главная особенность Jupyter Notebook заключается в том, что в одном документе можно одновременно размещать:

- программный код;
- текстовые пояснения;
- математические формулы;
- таблицы;
- графики;
- изображения;
- результаты вычислений.

Например, перед программной ячейкой можно описать цель вычисления, после неё вывести таблицу или график, а затем записать вывод. Такая последовательность делает ход решения понятным.

Благодаря этому вся работа над проектом находится в одном месте, что значительно упрощает разработку и анализ полученных результатов.

Почему именно Jupyter Notebook?

При разработке обычных программ программист чаще всего пишет код в одном окне, результаты получает в другом, а пояснения оформляет в отдельном документе.

При работе с анализом данных такой подход неудобен.

Например, если необходимо исследовать набор данных, разработчик обычно:

- загружает данные;
- пишет программу;
- строит графики;
- делает выводы;
- документирует результаты.

Если всё это хранить в разных файлах, становится сложно понять последовательность работы и повторить выполненное исследование.

Jupyter Notebook объединяет все эти действия в одном документе.

Ячейки позволяют выполнять исследование по этапам. Можно загрузить данные в одной ячейке, проверить их в следующей, затем выполнить вычисления и построить график, сохраняя пояснения рядом с каждым шагом.

Однако разделение на ячейки не отменяет зависимости между командами. Если следующая ячейка использует результат предыдущей, их необходимо выполнять в правильном порядке.

Поэтому данная среда разработки стала одним из основных инструментов специалистов по анализу данных и искусственному интеллекту.

Что такое ноутбук?

Каждый документ Jupyter называется

ноутбуком (Notebook)

Ноутбук представляет собой интерактивный документ, состоящий из отдельных блоков. Каждый блок называется

ячейкой

Интерактивность означает, что код можно запускать непосредственно в документе и сразу видеть результат рядом с выполненной ячейкой.

Все выполненные действия сохраняются внутри одного файла.

Файлы Jupyter Notebook имеют расширение

.ipynb

Расширение является частью имени файла и указывает, что документ предназначен для работы

в совместимой среде Jupyter. Простое переименование другого файла с добавлением

.ipynb

не превращает его в ноутбук.

Именно такие файлы вы будете создавать во время выполнения практических заданий.

Из чего состоит ноутбук

Каждый ноутбук включает несколько основных элементов.

Расположение отдельных кнопок может немного отличаться в разных версиях Jupyter, но назначение основных элементов остаётся тем же.

Строка меню

Содержит основные команды программы:

- создание файлов;
- сохранение;
- запуск вычислений;
- настройки;
- управление ноутбуком.

Через строку меню можно получить доступ как к часто используемым, так и к дополнительным действиям, связанным с ячейками и ядром.

Панель инструментов

Позволяет быстро выполнять наиболее распространённые действия.

Например:

- сохранить файл;
- создать новую ячейку;
- удалить ячейку;
- изменить её тип;
- выполнить код.

Перед удалением или изменением типа ячейки необходимо проверить, какая ячейка выбрана в данный момент. Команда применяется именно к активной ячейке.

Рабочая область

Это основная часть окна программы.

Именно здесь располагаются все ячейки вашего ноутбука.

В рабочей области можно выбирать ячейки, редактировать их содержимое, запускать код и просматривать результаты.

Название документа

Каждый ноутбук имеет собственное имя.

Лучше сразу использовать понятные названия файлов.

```
Lesson_01.ipynb
Homework_03.ipynb
Project.ipynb
```

Это значительно упростит поиск файлов в дальнейшем.

Название должно помогать определить содержание документа. Не рекомендуется хранить несколько разных работ под одинаковыми или случайными именами, поскольку при отправке легко выбрать не тот файл.

Ячейки — основа Jupyter Notebook

Вся работа в Jupyter Notebook строится вокруг ячеек.

Каждая ячейка представляет собой самостоятельный блок информации.

Самостоятельность ячейки означает, что её можно отдельно редактировать и запускать. При этом код внутри неё может использовать переменные и данные, созданные в других ячейках.

Ячейки можно:

- создавать;
- удалять;
- перемещать;
- копировать;
- изменять;
- выполнять независимо друг от друга.

Перемещение ячейки изменяет порядок отображения материала, но не отменяет уже выполненные команды, которые остаются в памяти ядра.

Существует два основных типа ячеек.

Ячейка Code

Ячейка

Code

предназначена для написания программ на языке Python.

После выполнения такой ячейки программа сразу отображает результат вычислений.

Результат обычно появляется непосредственно под ячейкой, поэтому можно быстро сопоставить команду и полученный вывод.

Например, после выполнения команды можно сразу увидеть:

- число;
- текст;
- таблицу;
- график;
- изображение;
- сообщение об ошибке.

Если ячейка не создаёт видимого вывода, это не всегда означает, что она не выполнилась. Например, команда могла сохранить значение в памяти для использования на следующем шаге.

Сообщение об ошибке также является результатом выполнения: оно показывает, что Jupyter попытался запустить код, но обнаружил проблему.

Ячейка Markdown

Не вся работа программиста состоит только из написания кода.

Очень важно уметь объяснять свои действия.

Для этого используется тип ячейки

Markdown

В такой ячейке можно создавать:

- заголовки;
- списки;
- таблицы;
- пояснения;
- ссылки;
- изображения;
- математические формулы.

Markdown-ячейки удобно использовать для постановки задачи, описания этапов работы и записи выводов после вычислений.

После выполнения Markdown превращается в красиво оформленный текст.

Чтобы изменить уже оформленный текст, необходимо снова перейти к редактированию

соответствующей ячейки, внести исправления и выполнить её повторно.

Благодаря этому один документ одновременно становится и программой, и отчётом.

Обратите внимание

Ячейка Markdown не выполняет Python-код. Если команда Python записана в Markdown-ячейке, она будет показана как текст и не произведёт вычислений.

Выполнение кода

После написания программы её необходимо выполнить.

До запуска содержимое Code-ячейки остаётся только записанным текстом и не влияет на состояние программы.

Для этого используется сочетание клавиш

Shift

Enter

Клавиши нажимаются одновременно: при удержании

Shift

нажимается

Enter

После выполнения происходит сразу несколько действий:

- программа запускается;
- отображается результат;
- активной становится следующая ячейка.

Если в коде есть ошибка, вместо ожидаемого результата появится сообщение с информацией о проблеме. После исправления ячейку необходимо выполнить ещё раз.

Именно этой комбинацией клавиш вы будете пользоваться практически на каждом занятии.

Что означают In и Out?

Слева от каждой программной ячейки можно увидеть обозначение:

```
In [ ]
```

Пустые скобки показывают, что ячейка ещё не выполнялась в текущем сеансе.

После выполнения программы пустые скобки заменяются номером.

Например:

```
In [1]
```

Это означает, что данная ячейка была выполнена первой.

Если программа выводит результат, под ней появляется обозначение:

```
Out [1]
```

Номер показывает последовательность выполнения программы.

Одинаковый номер у

In

Out

связывает введённую команду с результатом, полученным при её выполнении.

Важно понимать, что порядок расположения ячеек и порядок их выполнения могут отличаться.

Например, верхнюю ячейку можно выполнить после нижней. Тогда её номер окажется больше, хотя визуально она расположена раньше.

Из-за такого порядка ноутбук иногда работает в текущем сеансе, но выдаёт ошибку после перезапуска ядра. Это происходит, если нижняя ячейка зависит от команды, которая расположена выше, но была выполнена позже или не была выполнена совсем.

Поэтому иногда рекомендуется выполнять весь ноутбук заново.

Что такое Kernel?

Во время выполнения программы Jupyter Notebook использует специальный процесс, который называется

ядром (Kernel)

Ядро можно представить как рабочую память текущего сеанса: оно принимает команды из Code-ячеек, выполняет их и сохраняет созданные объекты.

Именно ядро выполняет весь написанный код.

Кроме этого, оно хранит в памяти:

- созданные переменные;
- результаты вычислений;
- загруженные данные;
- подключённые библиотеки.

Пока ядро работает, программа «помнит» всё, что было выполнено ранее.

Сохранение файла и состояние ядра — не одно и то же. В файле сохраняются ячейки и отображённые результаты, но после нового запуска ноутбука необходимые переменные обычно требуется создать заново, выполнив код.

Если же ядро перезапустить, память очищается, и выполнение программы начинается заново.

При этом записанный в ячейках код не удаляется. Очищается именно рабочее состояние: переменные, загруженные данные и другие объекты, созданные во время выполнения.

Именно поэтому иногда после открытия старого ноутбука некоторые ячейки могут перестать работать — сначала необходимо выполнить предыдущие команды.

Важно

Видимый результат под ячейкой может сохраниться в файле, даже если ядро уже не хранит использованные для него переменные. Поэтому наличие старого результата не доказывает, что ноутбук сейчас можно выполнить без ошибок.

Как правильно сохранять работу

Во время выполнения заданий рекомендуется регулярно сохранять ноутбук.

Хотя Jupyter автоматически сохраняет изменения через определённые промежутки времени, лучше периодически выполнять сохранение вручную.

Особенно важно сохранять работу после выполнения значимого этапа: добавления кода, исправления ошибки, построения графика или оформления вывода.

Перед отправкой задания обязательно убедитесь, что:

- все изменения сохранены;
- все ячейки выполнены;
- отсутствуют сообщения об ошибках;
- файл имеет правильное название.

Также следует проверить, что отправляется последняя версия файла. Если на компьютере сохранено несколько копий ноутбука, их названия должны позволять однозначно выбрать готовую работу.

После сохранения полезно обратить внимание на состояние документа и убедиться, что последние изменения действительно записаны.

Полезные рекомендации

Выполняйте ячейки последовательно

Не пропускайте предыдущие шаги.

Большинство программ зависит от результатов предыдущих вычислений.

Если ячейка использует переменную, которая должна быть создана выше, сначала необходимо выполнить ячейку с созданием этой переменной.

Не удаляйте готовый код

Если в задании уже подготовлена часть программы, изменяйте только те участки, которые указаны в инструкции.

Готовая часть кода может содержать необходимые настройки, загрузку данных или команды, от которых зависит дальнейшее выполнение задания.

Чаще сохраняйте работу

Это позволит избежать потери результатов при случайном закрытии программы или отключении компьютера.

Сохранение особенно важно перед перезапуском ядра, закрытием вкладки или отправкой файла преподавателю.

Внимательно читайте сообщения об ошибках

Сообщение об ошибке помогает определить причину проблемы.

Полезно обратить внимание на тип ошибки и строку, на которую указывает сообщение. Проверку следует начинать именно с этого места, учитывая, что настоящая причина иногда находится в предыдущей строке.

Не стоит сразу переписывать всю программу — часто достаточно исправить одну строку.

Не бойтесь экспериментировать

Программирование — это процесс постоянного поиска решений.

Даже опытные разработчики регулярно проверяют различные варианты, прежде чем получить окончательный результат.

Эксперименты лучше выполнять осознанно: изменять один элемент, запускать ячейку и сравнивать новый результат с предыдущим. Так легче понять влияние каждого изменения.

Совет

Перед отправкой работы преподавателю рекомендуется ещё раз выполнить весь ноутбук сверху вниз и убедиться, что все ячейки запускаются без ошибок.

Такая проверка показывает, что результат зависит от сохранённого кода и правильного порядка ячеек, а не от объектов, случайно оставшихся в памяти ядра.

Мини-проверка

Проверьте, удалось ли разобраться с основными элементами Jupyter Notebook:

- какой тип ячейки используется для запуска Python-кода;
- какой тип ячейки подходит для заголовков и пояснений;
- что происходит после нажатия Shift + Enter;
- почему номера выполнения могут не совпадать с порядком ячеек;
- какие данные теряются при перезапуске Kernel;
- что необходимо проверить перед отправкой файла.

Если на какой-либо вопрос пока нет точного ответа, следует вернуться к соответствующему разделу занятия.

Итоги занятия

Сегодня вы познакомились с организацией обучения и узнали, как будет проходить курс.

Теперь вы знаете:

- как организовано обучение на образовательной платформе;
- каким образом будут проверяться ваши задания;
- почему для выполнения практических работ используется Jupyter Notebook;
- что представляет собой ноутбук Jupyter;
- какие типы ячеек существуют;
- как выполнять код;
- что такое Kernel;
- как правильно сохранять работу и проверять ноутбук перед отправкой.

Главный результат первого занятия — готовность работать с материалами курса и использовать Jupyter Notebook как единое пространство для кода, пояснений и результатов вычислений.

Что дальше?

На следующем занятии вы перейдёте к изучению языка программирования

Python

и напишете свои первые программы. Именно с этого начнётся практическая часть курса и знакомство с основами программирования.